

Date : _____ Time : _____ Place : _____

Bug 1: Status filter ~~is~~ returning wrong tasks:

1. Where the bug was:

It was in the TaskRepository.java, @Query,

On the filter SearchTask(). This files exist in the backend part of the app inside where I made the changes was the SQL layer of the code.

2. How I found it:

At first, when I open/run the app and visited the app in the browser there is this dropdown status option which is used to filter the search, but the filter did not work properly, so I called the /api/tasks?status=OPEN directly in which I got 47 results but the database only has 31 OPEN tasks.

Date : _____ Time : _____ Place : _____

to verify this, I checked with Group by status in H2 console. It shows that some rows in the response ~~were~~ were DONE & IN-PROGRESS, It gets me clear that the bug is in the backend.

3 Root Cause:

The root cause was in the WHERE clause, In which AND and OR were mixed with no parentheses. Due to which SQL runs the AND before OR, ~~due to~~ which leads the query split into two halves. The title-matches half had no status check, so other statuses gets mixed up.

4. Fixed and Why:

- a. Put the two(2) LIKE conditions in one pair of parenthesis and joined everything else with plain AND due to this, now every row must pass all three checks, which are:

Date : _____

Time : _____

Place : _____

not archived, matches the search, and matches the status.

Why:

I matched/grouped the conditions with parentheses instead of restructuring the whole query, because the intended logic was already correct and only the precedence was wrong. That was the smallest change that helped in fixing the root cause.

I tested by restarting the backend and then checked that `/api/tasks?status=OPEN` returns only OPEN rows and that the total matches the count in H12 console i.e., 31.

Bug 2: Artificial delay in the Task API.

1. Where the bug is:

File name: backend/src/main/java/com/internal/
tasktracker/TaskController.java

Layer: Backend / Spring Boot Controller.

The issue is the Thread.sleep(queryWeight)
which call in the /api/tasks request.

This is used to calculate complexity score
from the search query length which was
being used to determine how long
request should sleep.

Date : _____

Time : _____

Place : _____

2. How I found it :

I was manually searching for a task using the search box/field and noticed that there was a noticable delay before the request was executed.

This made me investigate the reason behind the delay. While I was tracing the request flow, found a calculation that was adding an artificial delay based on the search query. Shorter queries were causing a larger delay, and search for an empty end up adding one second.

3. Root Cause:

The controller was using Thread.sleep() which was based on the calculated query complexity. The comment says it was intended for logging, but the implementation was blocking the request threads.

Date : _____ Time : _____ Place : _____

Due to this the server spends time waiting instead of processing the request, which increases response time & unnecessarily occupies the thread.

4. How I Fixed it & why:

Fixed it by removing the Complexity Score and Thread.sleep() completely.
As I found none of them is required for the actual task-search thread.

By doing this the artificial delay is removed and prevented the request thread to from being blocked unnecessarily.

Date : _____ Time : _____ Place : _____

Bug 3: Loading state does not stop when API request fails.

1. Where the bug is:

The bug was in the API request handling where, `setLoading(false)` was called only inside the `.then()` method.

File: frontend/src/hooks/useTasks.js

Layer: Frontend / React State management.

2 How I found it:

While, I disconnected the backend and came to check the UI part it still shows "loading" which made it unclear for the user if the backend is getting fetched or not.

3. Root Cause:

This happens when API request fails, the `.catch()` blocks sets the error message but does not reset the loading state.

The state becomes:

`loading = true,`

`error = <error message>`

In `TaskTable.jsx` it checks loading before checking for error, due to this the loading message is displayed and the error is never shown.

4 How I fixed it and why:

For this I moved `setLoading(false)` from the success handle to a `.finally()` block:

```
finally() => { setLoading(false); }
```

This ensures that the loading state is cleared whether the request succeeds or fails.

Date : _____

Time : _____

Place : _____

The reason behind choosing this approach is that finally () represents work that must happen after the request is completed regardless of its result.

It also prevents the loading state from becoming inconsistent in future error paths.